Functions

Activity

Work on the warmup activity (handout), then we will discuss as a class

Ordering and arguments

```r
my_power <- function(x, y){
  return(x^y)
}
```

```r
my_power(x = 2, y = 3)
```

```
[1] 8
```

```r
my_power(y = 3, x = 2)
```

```
[1] 8
```

```r
my_power(2, 3)
```

```
[1] 8
```

```r
my_power(3, 2)
```

```
[1] 9
```

- If you don't name the arguments when calling a function, R assumes you passed them in the order of the function definition

Function defaults

```
my_power <- function(x, y){
  return(x^y)
}
```

What will happen when I run the following code?

```
my_power(3)
```

Function defaults

```
1 my_power <- function(x, y){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power(3)
```

```
Error in my_power(3): argument "y" is missing, with no default
```

Function defaults

```
my_power <- function(x, y=2){
  return(x^y)
}
```

What will happen when I run the following code?

```
my_power(3)
```

Function defaults

default value of y if y is not specified by user

```
1 my_power <- function(x, y=2){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power(3)
```

```
[1] 9
```

Function defaults

```
1 my_power <- function(x, y=2){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power(2, 3)
```

2^2 ? 2^3 ?

Function defaults

```
1 my_power <- function(x, y=2){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power(2, 3)
```

```
[1] 8
```

2^3

use y=2 if user doesn't specify a value

otherwise, use user-specified value

Function defaults

```
my_power <- function(x, y){
  return(x^y)
}
```

What will happen when I run the following code?

```
my_power(3)
```

Function defaults

```
1 my_power <- function(x, y){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power(3)
```

```
Error in my_power(3): argument "y" is missing, with no default
```

Function defaults

```
1 my_power <- function(x=2, y=4){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power()
```

Function defaults

```
1 my_power <- function(x=2, y=4){
2   return(x^y)
3 }
```

What will happen when I run the following code?

```
1 my_power()
```

```
[1] 16
```

Other examples of defaults:

- ggplot aes options
- pnorm(..., lower.tail = TRUE) — default
- mean(..., na.rm = FALSE) — default

Function scoping

What value will the following code return?

```
g01 <- function(x = 10) {
  return(x)
}

g01()
```

Function scoping

What value will the following code return?

```
g01 <- function(x = 10) {
  return(x)
}

g01()
```

```
[1] 10
```

What if I try to look at x?

```
x
```

Function scoping

What value will the following code return?

```
g01 <- function(x = 10) {
  return(x)
}

g01()
```

```
[1] 10
```

What if I try to look at `x`?

```
x
```

```
Error: object 'x' not found
```

- Variables created within functions don't exist outside the function!

Function scoping

Variables created within functions don't exist outside the function!

```
g01 <- function() {
  x <- 10
  return(x)
}

g01()
```

```
[1] 10
```

```
x
```

```
Error: object 'x' not found
```

Function scoping

What will the following code return?

```
x <- 10

g01 <- function(){
  return(x)
}

g01()
```

Function scoping

```r
x <- 10

g01 <- function(){
  return(x)
}

g01()
```

```
[1] 10
```

```r
x
```

```
[1] 10
```

- If a variable is not defined in a function, R looks outside the function (the *global environment*)

Name masking

What value will the following code return?

```
x <- 10
g01 <- function() {
  x <- 20
  return(x)
}

g01()
x
```

Name masking

What value will the following code return?

```
1 x <- 10
2 g01 <- function() {
3   x <- 20
4   return(x)
5 }
6
7 g01()
```

```
[1] 20
```

x defined inside function masks x defined outside function

But, x define inside function does not exist outside, and does not change or global x

```
1 x
```

```
[1] 10
```

- Names defined inside a function *mask* names defined outside a function
- Variables created within a function don't exist outside

Summary

- Variables created within a function don’t exist outside
- If a variable is not defined in a function, R looks outside the function
- Names defined inside a function *mask* names defined outside a function

Class activity

Work on the class activity (course website).